How does matrix size affect the runtime and memory of naive matrix multiplication in Python, Java and C?

Big Data - Individual Assignment 1: Basic matrix multiplication in different languages
Grado en Ciencia e Ingeniería de Datos, Universidad de Las Palmas de Gran Canaria, 2026-2027
Author: Guillermo Oltra Rodriguez - **Date:** October 2026
Materials: repository folder `practica1/` (code, inputs, raw measurements, analysis scripts; see `README.md` and Section 8)

> **Abstract.** This study compares the basic triple-loop multiplication of dense `n x n` matrices of `float64` numbers written in Python 3.11, Java 17 and C (gcc 11, `-O2`). The three programs use the same loop order and read the same input files, and all runs were done on the same laptop (AMD Ryzen 7 4800H, Windows 11). I measured 14 sizes from 10 to 2000, with a new process for every measurement and 5 repetitions. The three programs give the same result. Python is about 100 times slower than C (102 to 117 times for 100 <= n <= 800). Java is almost as fast as C for small matrices (1.1 times) and 1.5 to 2.7 times slower for large ones. The runtime grows roughly as n^3, but a bit faster (slopes of 3.2 to 3.6 in a log-log plot), because the column-wise access to `B` stops fitting in the CPU cache as n grows. Memory is a fixed cost of the runtime (C about 4 MB, Python about 23 MB, JVM about 60 MB) plus a part that grows with n^2 and is about five times larger in Python. With a limit of 300 s per run, Python reaches n = 1000 and Java and C reach n = 2000. Two extra experiments look at compiler/JIT settings and at more sizes. The conclusions only apply to these implementations, settings and machine.

1. Introduction

Matrix multiplication is a basic operation in numerical computing and machine learning. The simple triple loop is the starting point for the rest of the course assignments (optimised, sparse, parallel and distributed versions), so it is useful to first understand how it behaves in different languages. I chose Python, Java and C because they run code in very different ways: Python is interpreted, Java uses a JIT compiler, and C is compiled before running.

Research question. How do the runtime and the memory use of the naive triple-loop multiplication grow with the matrix size `n` when it is written in the same way in Python, Java and C, and how close is that growth to the theoretical $O(n^3)$?

- **What changes:** the size `n` (14 values from 10 to 2000) and the language.
- **What is measured:** time per multiplication (median and IQR of 5 repetitions) and peak memory of the process.
- **What stays the same:** the algorithm and loop order (`i-j-k`), the numeric type (`float64`), the input files, the machine and what the timer includes.

Hypotheses (written before the main campaign, based on a first Python-only test run, `results/01_oficial_py311`):

- **H1.** For `n >= 100` the time will grow roughly as n^3 (slope about 3 in a log-log plot) in the three languages, maybe a bit more for large `n` because the inner loop reads `B` by columns, which is bad for the cache.
- **H2.** Python will be around 100 times slower than C, because of the interpreter (about 70 ns per inner iteration in the test run, against about 1 ns expected in C).

- **H3.** After warm-up, Java will be close to C (between 1 and 3 times slower).
- **H4.** For small `n` memory will be dominated by the runtime (C a few MB, Python about 20 MB, JVM tens of MB). The part that grows with `n` will be about 3 x 8 x n^2 bytes in C and Java and about 3 x 32 x n^2 bytes in Python, because each Python number is a separate `float` object.

2. Background

Algorithm. For `C = A x B` with `n x n` matrices, the triple loop computes each element `C[i][j]` as the sum of `A[i][k] * B[k][j]` for `k` from 0 to n-1 [1]. This needs n^3 multiplications and additions. If every iteration of the inner loop took the same time, the total time would be `t(n) = c * n³` and the log-log plot of time against `n` would be a straight line with slope 3. If the slope is different, it means that the cost per iteration `c` changes with `n`.

Cache and loop order. The CPU keeps recently used data in small and fast memories called caches (L1, L2 and L3), and reading data that is not in cache is much slower [2]. In the `i-j-k` order the inner loop reads a row of `A`, which is contiguous in memory, but it reads a column of `B` (`B[0][j], B[1][j], ...`), so two consecutive reads are `8n` bytes apart. When `B` is small it stays in cache, but for large `n` it does not fit anymore and each iteration becomes more expensive. On the Ryzen 7 4800H used here, each core has 32 KiB of L1 data cache and 512 KiB of L2, and each group of 4 cores shares 4 MiB of L3 [3].

How each language runs the code. CPython interprets bytecode, and every number in a list is a separate `float` object, so even a simple `a * b` has a lot of overhead. Java first interprets the bytecode and then the JIT compiler turns the most used methods into machine code while the program is running [7], so it needs some warm-up before it reaches full speed. C is compiled to machine code before running, and the compiler flags control how much it optimises [5].

3. Methodology

3.1 Implementations

The three implementations run the same statement with the same loop order. No matrix library is used inside the timed code; NumPy is only used to check the results.

```
# Python (src/python/matmul.py)
def matmul_ijk(A, B, n):
    C = [[0.0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

Java (`src/java/MatMul.java`) and C (`src/c/matmul.c`) use exactly the same three loops and the same statement `C[i][j] += A[i][k] * B[k][j]`. Each language uses its usual data structure, which is a difference that cannot be avoided:

Language	Data structure	Storage per matrix	Version and settings
Python	list of lists of `float` objects	about 32 n^2 bytes (24 B object + 8 B pointer)	CPython 3.11.9
Java	`double[][]` (array of separate `double[]` rows)	about 8 n^2 bytes + 16 B per row	Temurin 17.0.20.1

Language	Data structure	Storage per matrix	Version and settings
C	one contiguous block of `n²` doubles, row by row	8 n^2 bytes	gcc 11.2.0 `-std=c11 -O2 -ffp-contract=off`

In C I used `-ffp-contract=off` [5] so that gcc does not merge the multiplication and the addition into one instruction. Without it, C could round slightly differently from Python and Java.

3.2 Inputs

The three programs read **the same binary files** `data/A_n.bin` and `data/B_n.bin` (`float64`, row by row), created once by `src/python/datagen.py`. Values are random numbers between 0 and 1, generated with Python's `random` module with a fixed seed for each size (base seed 2026). The seeds and the SHA-256 hash of each file are saved in `data/manifest.json`, and the files are checked before each campaign. Using shared files avoids the problem that each language has a different random generator.

3.3 Environment

Item	Value
Machine	Laptop, AMD Ryzen 7 4800H, 8 cores / 16 threads, 2.9 GHz (boost up to 4.2 GHz)
Caches [3]	L1d 32 KiB and L2 512 KiB per core; L3 2 x 4 MiB
Memory	15.4 GiB
Operating system	Windows 11 Home (build 26300)
Power	Plugged in, default power plan
Python	CPython 3.11.9, garbage collector disabled during the timed part
Java	OpenJDK 17.0.20.1 (Temurin), `-XX:+UseSerialGC -Xms2g -Xmx2g` [6], default JIT
C	gcc 11.2.0 (MinGW-w64), `-std=c11 -O2 -ffp-contract=off`
Threads	One computing thread in the three languages
Other libraries	NumPy 2.4.6 (only for checking), matplotlib 3.11.2, psutil 7.2.2

This information is saved automatically in `results/*/environment.md` and `build/build_info.json`. For Java I used one GC thread and a fixed heap of 2 GB, so that no garbage collection happens during the measured part. The JVM still creates around 20 threads of its own (JIT, GC and others), but the multiplication runs in one thread.

3.4 What the timer includes

Only the multiplication function: creating `C`, setting it to 0.0 and running the three loops. Starting the interpreter or JVM, reading the files, the warm-up and checking the result are **outside** the timer. I used timers meant for measuring short durations: `time.perf_counter_ns()` in Python [4], `System.nanoTime()` in Java and `QueryPerformanceCounter` in C. The result is always used afterwards (a checksum of all elements), so the compiler cannot skip the computation.

3.5 Measurement protocol

- **A new process for each repetition** and language, so every measurement starts from the same state.
- **Interleaved order.** For each `n` and repetition the three languages run one after the other, and the order changes in each repetition, so slow changes in the laptop (temperature, background tasks) do not favour one language.
- **5 repetitions** per configuration. If the first one takes more than 120 s only 3 are done; this only happened for Python at `n = 1000`.

- **Short tasks.** If one multiplication takes less than 100 ms, a batch of several multiplications is timed together and the time is divided by the batch size.
- **Warm-up** (not included in the results). Python and C: one run on a 64x64 matrix. Java: 200 runs on 64x64 matrices, so the JIT has compiled the method before the timer starts. My first version used 5 runs of 256x256, but at `n = 1000` the times changed a lot between runs (about 4.0 s in most runs and 2.4 s in one), so I changed it. Both versions were measured in `results/05_extra_variantes`.
- **Statistics.** Median and inter-quartile range (IQR). All raw measurements are kept in `raw.csv` and none was removed.

3.6 Memory metric

The memory compared between languages is the **peak working set of the whole process** (the maximum physical memory it used), read by the benchmark script with the Windows function `GetProcessMemoryInfo` when the process ends. It is measured in the same way for the three languages and includes the interpreter, the JVM or the C runtime. Each program also records its own memory before and after the multiplication, to see how much the matrices add. In Java this second value is the used heap, which is a different quantity, so it is not compared with the others. I also compare with a simple storage estimate ($3 \times 8 \times n^2$ bytes for three dense matrices).

3.7 Correctness

Before measuring, **36 automated tests** (`tests/`) are run: small cases computed by hand (1x1, 2x2, 3x3), identity and zero matrices, non-commutativity (to detect swapped indices), comparison with NumPy and tests of the file reading, the data generator and the C and Java programs.

During the campaign, in repetition 1 of each language and size, the result `C` is saved after the timer and compared with NumPy's `A @ B` using a relative tolerance of 1e-9 and an absolute tolerance of 1e-12. NumPy adds the products in a different order, so tiny differences (around 1e-15) are expected; the tolerance is much larger than that, but a wrong index would give errors of order 1, so it would still be detected. I also saved the SHA-256 hash of each result to check whether the three languages give exactly the same numbers. In repetitions 2 to 5 the checksum must be the same as in repetition 1.

3.8 Sizes and limits (decided before measuring)

- **Sizes:** 10, 25, 50, 100, 200, 300, 400, 500, 600, 800, 1000, 1200, 1500, 2000.
- **Time limit:** 300 s per process. **Memory limit:** 4096 MB per process.
- If a run goes over a limit, it is recorded as a timeout (with no time) and that language does not try larger sizes.

4. Results

The data are in `results/04_comparativa_3_lenguajes/`: `raw.csv` (194 runs: 193 correct and 1 timeout), `summary.md` (complete tables), `campaign.log` and `plots/`. The campaign took 27 minutes (14 Oct 2026), in one session, with the laptop plugged in.

4.1 Correctness

- The 36 tests passed before measuring.
- In every language and size the result matches NumPy within tolerance (maximum relative error 5.6e-15).
- **The result `C` is exactly the same (same SHA-256) in Python, Java and C for all 14 sizes.** So the three programs do the same operations in the same order, and the time difference only comes from how each

language runs them.
- In repetitions 2 to 5 the checksum always matched repetition 1.

4.2 Runtime

Median time per multiplication (5 repetitions; 3 for Python at `n = 1000`). In brackets, IQR/median when it is above 3 %.

n	Python	Java	C (-O2)	Python / C	Java / C
10	82.2 µs	1.09 µs	0.441 µs	186x	2.46x
25	1.13 ms	11.1 µs	6.34 µs	178x	1.75x
50	8.53 ms	81.0 µs	59.5 µs	143x	1.36x
100	67.6 ms	661 µs	599 µs	113x	1.10x
200	0.539 s	5.74 ms	5.28 ms	102x	1.09x
300	1.90 s	27.5 ms	18.6 ms	102x	1.48x
400	4.70 s	70.6 ms (4.2 %)	45.1 ms	104x	1.56x
500	9.87 s	154 ms (25 %)	89.3 ms	110x	1.72x
600	18.3 s	259 ms (9.3 %)	156 ms	117x	1.66x
800	49.4 s	1.19 s	0.472 s	105x	2.51x
1000	181.6 s (3 reps)	2.46 s (3.6 %)	0.934 s (4.1 %)	194x	2.63x
1200	**timeout (> 300 s)**	4.20 s	1.61 s	-	2.60x
1500	not attempted	8.24 s	4.60 s	-	1.79x
2000	not attempted	23.4 s	10.8 s	-	2.17x

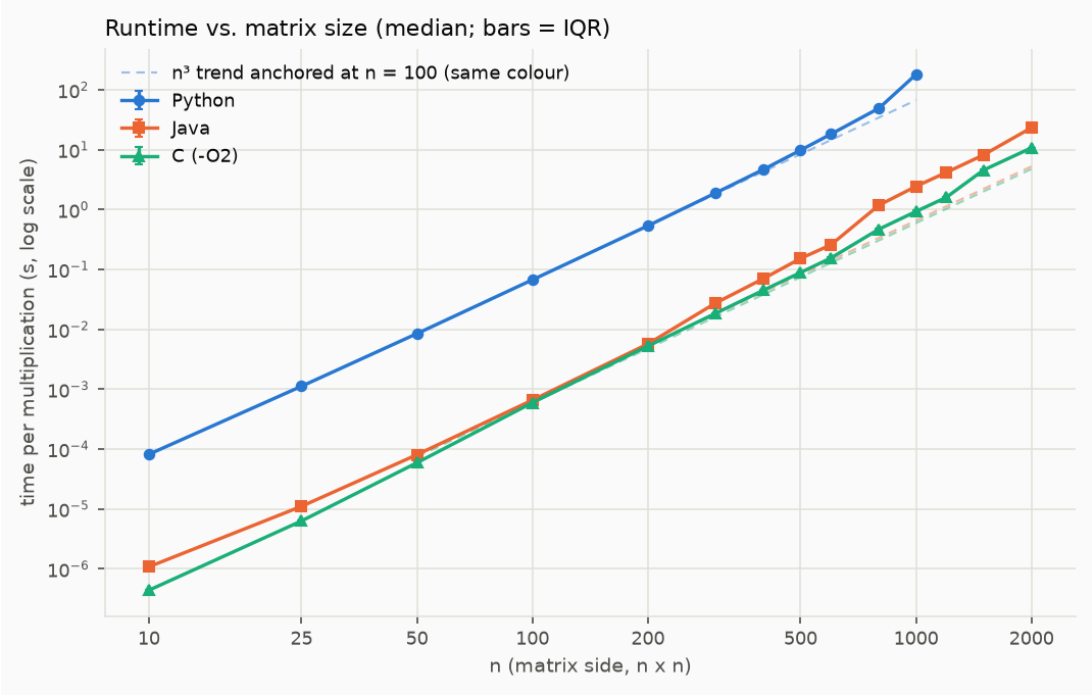


Figure 1. Time per multiplication against n (log-log; median with IQR bars, usually smaller than the marker). Dashed lines: n^3 trend starting at n = 100.

Variability. IQR/median is below 2 % in 31 of the 39 configurations and never above 4.2 %, except Java at `n = 500` (25 %) and `n = 600` (9 %). In those two cases most repetitions are close together, but one or two are much

slower. C, which ran in the same rounds, does not show this, so it seems to come from Java itself and not from noise on the machine.

Limits. With the 300 s limit, the largest size Python completed is **n = 1000**. At `n = 1200` the first repetition reached the limit, so it is recorded as a timeout and larger sizes were not tried. Java and C completed all sizes up to **n = 2000** with no timeouts or memory problems. Python at `n = 1000` has only 3 repetitions (the first took 184 s), but they are very close (180.8 to 182.9 s).

4.3 Growth compared with n^3

Slope of log(time) against log(n) (least squares):

Range	Python	Java	C
100 <= n <= 600 (800 for Python)	3.16	3.39	3.11
100 <= n <= largest size	3.32	3.57	3.28

The three slopes are close to 3 but all a bit higher. Figure 2 shows the time divided by n^3 (time per inner iteration), which should be a flat line if the time were exactly proportional to n^3:

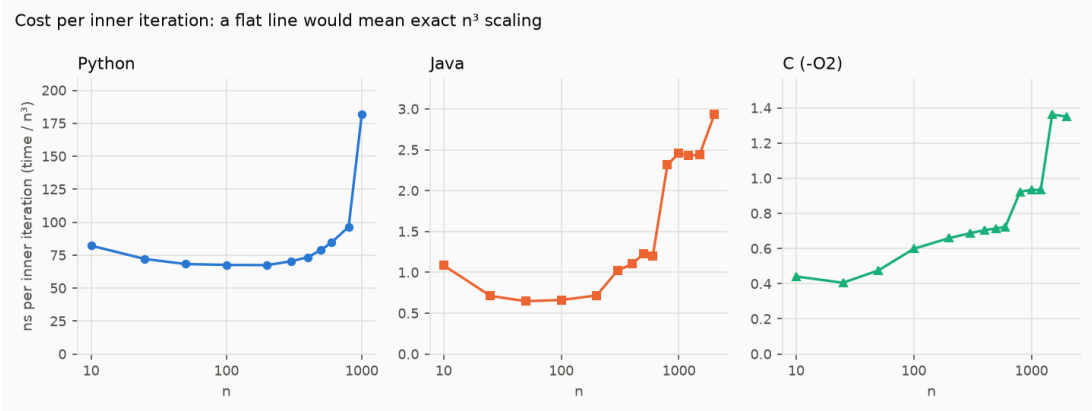


Figure 2. Time per inner iteration (time / n^3) for each language, each with its own vertical scale.

- **C:** about 0.4 to 0.7 ns per iteration up to `n = 600`. Then two jumps: 0.72 to 0.92 ns between `n = 600` and `800`, and 0.93 to 1.36 ns between `1200` and `1500`. Between jumps the cost is almost constant.
- **Java:** about 0.65 to 0.72 ns up to `n = 200` (almost the same as C), about 1.0 to 1.2 ns between 300 and 600, and a jump to about 2.3 to 2.5 ns between 600 and 800.
- **Python:** about 67 to 70 ns between 50 and 300, rising slowly to 96 ns at `n = 800` and then a big jump to 182 ns at `n = 1000`. This jump also appeared in the first test run and in the 3 repetitions here, so it is not random noise.

4.4 Comparison between languages

The last two columns of the table in Section 4.2 give the factors (plot in `results/04_comparativa_3_lenguajes/plots/slowdown_vs_n.png`):

- **Python / C:** between **102 and 117 times** for `100 <= n <= 800`. It is higher for very small `n` (186 times at `n = 10`) and at `n = 1000` (194 times, because of the Python jump).
- **Java / C: 1.1 times** for `n = 100` to `200` and between **1.5 and 2.6 times** for `n >= 300`. For very small `n` Java is also slower (1.8 to 2.5 times), probably because creating `double[n][n]` (n + 1 objects) is expensive compared with so few operations.

4.5 Memory

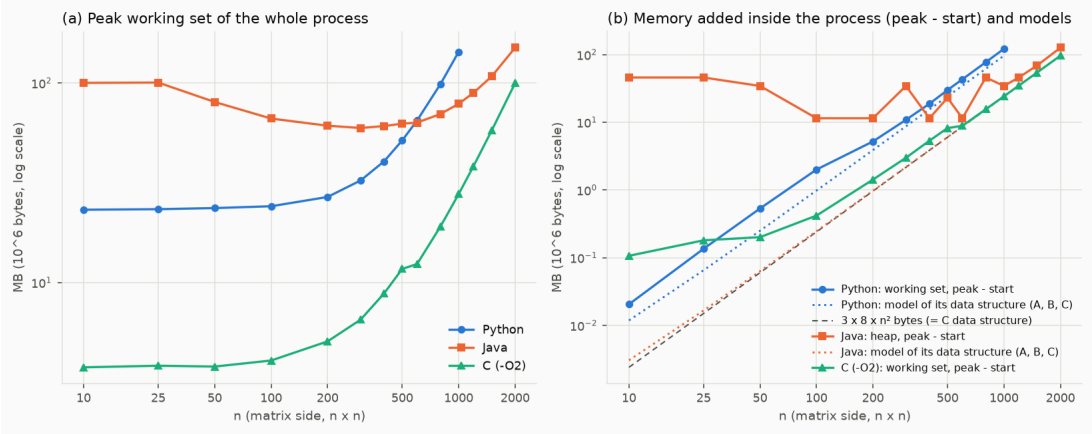


Figure 3. (a) Peak working set of the whole process, measured in the same way for the three languages. (b) Memory added inside the process by the matrices, with the storage estimates.

n	Python	Java	C	Estimate 3 x 8 x n^2
10	23.2 MB	100.1 MB	3.8 MB	0.002 MB
100	24.1 MB	66.3 MB	4.1 MB	0.24 MB
300	32.5 MB	59.5 MB	6.5 MB	2.2 MB
600	64.8 MB	63.3 MB	12.4 MB	8.6 MB
1000	143.2 MB	78.6 MB	27.7 MB	24.0 MB
2000	-	150.1 MB	99.7 MB	96.0 MB

Peak working set of the whole process (median of 5 runs; MB = 10^6 bytes).

- **Fixed cost of the runtime** (small n): C about 3.8 MB, Python about 23 MB and the JVM about 60 MB. Java uses *more* memory at `n = 10` to `50` (80 to 100 MB) than at `n = 300`. This is because of batching: at `n = 10` about 48 000 multiplications run one after the other, each one creating a new `C`, and with a 2 GB heap the garbage collector never runs, so the garbage accumulates.
- **Part that grows with n^2.** In C the memory added by the matrices is almost exactly the estimate: 96.2 MB measured against 96.0 MB at `n = 2000`. In Java it is similar (8 bytes per element plus a small header per row). In Python the matrices add 121.6 MB at `n = 1000`, about **5 times** more than in C, because each element is a pointer to a separate `float` object.
- So **for small n the runtime dominates** (Java > Python > C) and **for large n the data dominate** (Python > Java and C). The change happens around `n = 600`.

4.6 Extra experiment: compiler and JIT settings

This experiment is **kept separate** from the main comparison, as the assignment asks. It changes one setting at a time to see how much it matters (`results/05_extra_variantes/`, 5 repetitions, with the normal C and Java configurations measured again in the same runs, which repeat the main results within 2 %). All variants give the same result `C`.

n	C -O0	C -O2 (base)	C -O3 -march=native	Java -Xint	Java (base)
10	3.31 µs (7.4x)	0.450 µs	0.496 µs (1.10x)	24.9 µs (55x)	1.10 µs (2.4x)
100	3.16 ms (5.2x)	0.604 ms	0.613 ms (1.01x)	22.2 ms (37x)	0.677 ms (1.1x)
400	203 ms (4.5x)	45.1 ms	45.3 ms (1.00x)	1.40 s (31x)	69.0 ms (1.5x)
1000	4.16 s (4.5x)	0.925 s	0.935 s (1.01x)	25.5 s (28x)	2.47 s (2.7x)

Median time per multiplication; in brackets, times slower than C -O2 in the same campaign. Full table in `results/05_extra_variantes/summary.md`.

- **C optimisation level.** Without optimisation (`-O0`) C is 4.5 to 7.4 times slower. `-O3 -march=native` gives no improvement over `-O2` [5]. A likely reason is that the inner loop is a running sum, and gcc does not vectorise it unless it is allowed to change the order of the additions (`-ffast-math`), which would change the result.
- **Java JIT.** With `-Xint` (interpreter only, no JIT) [6] Java is 28 to 55 times slower than C. So almost all of Java's speed in the main comparison comes from the JIT.

4.7 Extra experiment: more sizes in C and Java

To see where the jumps of Figure 2 happen, C and Java were measured at n = 400, 450, ..., 1600 (every 50) plus 512, 1024 and 1536 (`results/07_extra_barrido_fino/`, 3 repetitions, same protocol and checks).

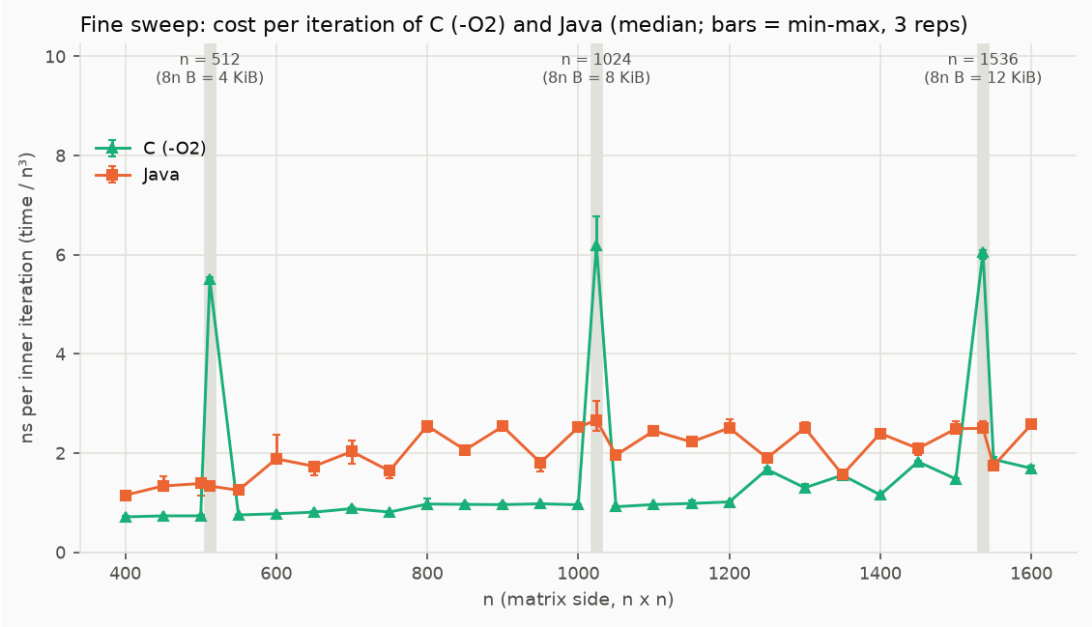


Figure 4. Time per inner iteration of C and Java with more sizes. Grey bands mark the multiples of 512.

- **Powers of two.** At n = 512, 1024 and 1536, C becomes 6 to 7 times slower than at nearby sizes (5.5 to 6.2 ns per iteration against 0.73 ns at n = 500 and 0.96 ns at n = 1000). Java does not have this problem, so at n = 1024 Java is 2.3 times *faster* than C, while at n = 1000 it is 2.6 times *slower*.
- **C** stays at about 0.71 to 0.81 ns up to n = 750, about 0.92 to 1.02 ns between 800 and 1200, and above 1250 it varies between 1.2 and 1.9 ns without a clear pattern.

5. Discussion

5.1 Python: the interpreter is the main cost (H2 confirmed)

Since the three programs give exactly the same result (Section 4.1), they do the same floating-point operations; the difference is how long each language takes to execute them. In Python each inner iteration `C[i][j] += A[i][k] * B[k][j]` needs many bytecode instructions: several list accesses, the multiplication and the addition (each one creating a new `float` object), plus type checks and reference counting. This takes about 70 ns per iteration, while C does it in about 0.4 to 0.7 ns with a few machine instructions. That is why Python is about 100 times slower, and since both do n^3 iterations the factor stays almost constant between n = 100 and 800.

Python's data structure also uses more memory: each element of a list is a pointer to a separate `float` object, so the matrices take about 5 times more memory than in C (Section 4.5). This probably explains why Python's cost per iteration starts to grow earlier (from n = 300), when its three matrices (about 96 n^2 bytes, 8.6 MB at n = 300) no longer fit in the L3 cache.

The big jump at n = 1000 (182 ns per iteration against 96 ns at n = 800) is **not explained** by this study. It happens in all 3 repetitions and also in the first test run, so it is not random, but I did not investigate the cause. It could be related to how the `float` objects are placed in memory.

5.2 Java vs. C: close while the data fit in cache (H3 confirmed)

At n = 100 to 200, when the three matrices fit in the cache, Java takes only 1.1 times the time of C, so the JIT generates code almost as good as gcc's. The `-Xint` experiment (Section 4.6) shows that this is thanks to the JIT. For larger n Java is 1.5 to 2.7 times slower. A possible reason is the `double[][]` structure: each row is a separate object, so reading `B[k][j]` first needs the reference to row `B[k]` and Java also checks that `j` is inside the array. This adds more memory accesses in the column walk. I did not measure this directly, so it is only a likely explanation.

5.3 Why the time grows faster than n^3 (H1 partly confirmed)

The number of iterations is exactly n^3 in the three languages, so a slope above 3 means that each iteration becomes more expensive as n grows. The reason is the access to `B` by columns: each read of `B[k][j]` is 8n bytes after the previous one, so it is in a different cache line, and that line is only useful again if it is still in cache for the next columns [2].

- **While everything fits in cache** (C with n <= 600) the n^3 model works well (slope 3.11).
- **The first C jump, between n = 600 and 800,** happens when `B` (8 n^2 bytes) stops fitting in the 4 MiB L3 cache: `B` takes 3.9 MB at n = 700 and 5.1 MB at n = 800. After that, `B` has to be read from main memory again for each row of `C`.
- **Powers of two (n = 512, 1024, 1536).** In this case the distance between `B[k][j]` and `B[k+1][j]` (8n bytes) is a multiple of 4096 bytes. Because of how the cache decides where to store each address, all the elements of a column go to the same small part of the cache and keep replacing each other (cache conflicts) [2]. Java does not have this problem because each row starts with a 16-byte header, so the distance is not an exact power of two. This shows that the memory layout can matter more than the language: at n = 1024 Java is faster than C.
- **Python** is affected earlier and more gradually, as explained in Section 5.1.

So $O(n^3)$ describes the **amount of work** well, but the **time** only follows n^3 while the data stay in the same cache level. The slopes of 3.2 to 3.6 are not a different algorithm, but the effect of the memory hierarchy.

5.4 Memory (H4 confirmed)

The results show the two parts I expected (Section 4.5): a fixed cost of the runtime, which dominates for n <= 600, and a part that grows with n^2 and follows the storage estimate of each structure. The working set only counts memory that is really used, so the 2 GB reserved for the Java heap does not appear.

5.5 Limitations

- **One laptop.** The CPU frequency could not be fixed (boost and temperature change it). To reduce this I used a new process per run, interleaved order, medians and IQR, and the laptop was plugged in. The variability was low (IQR below 2 % in 31 of 39 configurations), but the absolute times and factors could be different on another machine.

- **Background activity.** The system was not isolated (Windows services, OneDrive syncing the project folder). The interleaved order spreads this over the three languages.
- **Batches for small n.** For small sizes the time is the average of several multiplications in a row, with the cache already warm, not the time of a single call.
- **Fixed settings.** Java depends on the JVM settings (GC, heap, warm-up) and C on the compiler flags; Section 4.6 shows that these can change the results a lot. I used reasonable, documented settings, not the best possible ones.
- **No hardware counters.** The explanations about cache are based on the times and the sizes of the data, not on direct measurements of cache misses.

6. Conclusions

These conclusions only apply to these implementations (`i-j-k` triple loop, `float64`), these settings (CPython 3.11.9; Temurin 17 with `-XX:+UseSerialGC -Xms2g -Xmx2g` and the described warm-up; gcc 11.2 `-O2 -ffp-contract=off`), these inputs and this laptop. They do not show that one language is faster in general.

1. **Correctness.** The three programs compute exactly the same result at every size, so the time differences only show how each language runs the same computation.
2. **Python vs. C.** Python is about 100 times slower (102 to 117 times for 100 <= n <= 800) because of the cost of interpreting each iteration (about 70 ns), not because of the arithmetic.
3. **Java vs. C.** With a good warm-up, Java is within 1.1 times of C while the data fit in cache and 1.5 to 2.7 times slower for large n. But at powers of two (512, 1024, 1536) C has cache conflicts and Java is up to 2.3 times faster, so there is no fixed ranking.
4. **Growth.** The time follows n^3 approximately (slope 3.11 for C while everything fits in cache), but over the full range it grows faster (3.2 to 3.6) because the cost per iteration increases when `B` no longer fits in the cache.
5. **Memory.** There is a fixed runtime cost (C about 3.8 MB, Python about 23 MB, JVM about 60 MB) plus a part that grows with n^2, about 5 times larger in Python.
6. **Limits.** With 300 s per process, the largest size is n = 1000 for Python (timeout at n = 1200) and n = 2000 for Java (23 s) and C (11 s). There were no memory failures (highest peak: 150 MB).

What remains unresolved.

- Why Python jumps to almost twice the cost per iteration at n = 1000.
- Why C's cost varies without a clear pattern above n = 1250.
- How much of the 1.5 to 2.7 times gap between Java and C comes from the `double[][]` structure and the index checks. Measuring cache misses with hardware counters would help to answer these questions.
- Whether the results are the same on other machines or operating systems.

7. References

[1] Wikipedia, "Matrix multiplication algorithm." [Online]. Available: https://en.wikipedia.org/wiki/Matrix_multiplication_algorithm (accessed Oct. 15, 2026).

[2] Wikipedia, "CPU cache." [Online]. Available: https://en.wikipedia.org/wiki/CPU_cache (accessed Oct. 15, 2026).

[3] 7-cpu.com, "AMD Zen2." [Online]. Available: https://7-cpu.com/cpu/Zen2.html (accessed Oct. 15, 2026).

[4] Python Software Foundation, "time - Time access and conversions," *The Python Standard Library*, Python 3.11 documentation. [Online]. Available: https://docs.python.org/3.11/library/time.html (accessed Oct. 15, 2026).

[5] Free Software Foundation, "Options That Control Optimization," *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html (accessed Oct. 15, 2026).

[6] Oracle, "The java command," *Java Development Kit Version 17 Tool Specifications*. [Online]. Available: https://docs.oracle.com/en/java/javase/17/docs/specs/man/java.html (accessed Oct. 15, 2026).

[7] Wikipedia, "Just-in-time compilation." [Online]. Available: https://en.wikipedia.org/wiki/Just-in-time_compilation (accessed Oct. 15, 2026).

8. Materials and reproducibility

Everything can be regenerated from the `practica1/` folder. `README.md` has the exact commands to build, run each campaign and make the tables and plots, and the versions used. The main campaign is run with:

```
py -3.11 -m pip install -r requirements.txt
py -3.11 scripts/run_benchmarks.py --out results/04_comparativa_3_lenguajes/raw.csv
```

Item	Location
Source code	`src/python/matmul.py`, `src/java/MatMul.java`, `src/c/matmul.c`
Build and flags	`scripts/build.py`, `build/build_info.json`
Input data	`src/python/datagen.py` + `data/manifest.json` (seeds and SHA-256)
Tests (36)	`tests/test_matmul.py`, `tests/test_workers.py`
Benchmark script	`scripts/run_benchmarks.py`
Raw measurements (one row per run)	`results/*/raw.csv`
Environment of each campaign	`results/*/environment.md`
Tables and plots	`scripts/analyze.py`, `scripts/plot_extras.py`: `results/*/summary.md`, `plots/`

Each row of the CSV files includes the size, language, variant, numeric type, threads, repetition, batch, warm-up, runtime version and flags. No measurement was removed. The folder `01` is the first Python-only test run (see `results/LEEME.md`). Code comments and `results/LEEME.md` are in Spanish.

Use of generative AI. As required by the course policy, the author discloses that a generative AI assistant (Claude Code, Anthropic) was used as a support tool in three ways: to clarify concepts (for example JIT compilation, CPU caches and floating-point behaviour), to help with the code (the C and Java versions, the measurement scripts and the tests), and to help with writing and translating the report. The author is responsible for the content of this report and for defending it.